

First Come First Serve

Header line includes the standard input/output library, which allows the use of functions like printf and scanf to handle input and output operations.

A struct (structure) named Process is defined. This structure will hold information related to a process in a scheduling algorithm. The structure contains the following fields:

- pid: An integer representing the Process ID.
- arrival: An integer representing the Arrival Time of the process (when it arrives in the queue).
- burst: An integer representing the Burst Time (the time required for the process to complete its execution).
- completion: An integer representing the Completion Time (when the process finishes execution).
- turnaround: An integer representing the Turnaround Time (time from arrival to completion).
- waiting: An integer representing the Waiting Time (time spent waiting in the queue).

Function Name: sortByArrival

- Purpose: Sorts the array of processes (proc[]) based on their arrival time.
- Parameters:
- proc[]: An array of Process structures.
- n: The number of processes in the array.
 - **Working:**
 - Two nested for loops are used to compare the arrival times of each pair of processes.
 - If the arrival time of the process at index i is greater than the arrival time of the process at index j, they are swapped.
 - This ensures that the processes are sorted in ascending order of their arrival times.

Function Name: firstComeFirstServe

- Purpose: Implements the First-Come-First-Serve (FCFS) scheduling algorithm.
- Parameters:
- proc[]: An array of Process structures.
- n: The number of processes in the array.
 - **Variables:**
 - currentTime: Tracks the current time as processes are executed.
 - totalTurnaround and totalWaiting: Accumulators to calculate the total Turnaround and Waiting times for all processes, which will later be used to compute averages.

Process Execution Loop:

- The loop iterates over each process in the array proc[].
- Condition: If the currentTime is less than the arrival time of the process, it is set to the arrival time, simulating waiting until the process arrives.
- Completion Time Calculation: completion is calculated as the sum of currentTime and the process's burst time.
- Turnaround Time Calculation: turnaround is calculated as the difference between completion and arrival times.
- Waiting Time Calculation: waiting is calculated as the difference between turnaround and burst times.

- The totalTurnaround and totalWaiting variables are updated with the respective times for each process.
- The currentTime is updated to the completion time of the current process, preparing for the next iteration.

Display Execution Sequence:

This loop prints out the sequence in which the processes were executed based on their Process IDs (pid).

Display Process Details:

This section prints a table of the process details: PID, Arrival, Burst, Completion, Turnaround, and Waiting times.

Calculate and Display Averages:

The average Turnaround Time and Waiting Time are calculated and displayed.

These averages are obtained by dividing the totalTurnaround and totalWaiting by the number of processes (n).

Main Function:

Variables:

n: An integer to store the number of processes.

proc[100]: An array of Process structures to hold up to 100 processes.

User Input:

Prompts the user to input the number of processes (n), which is then stored.

Process Information Input:

This loop iterates over each process, assigns it a pid (starting from 1), and then prompts the user to input the arrival and burst times for each process.

Sorting and Scheduling:

Calls sortByArrival to sort the processes by their arrival time.

Calls firstComeFirstServe to execute the FCFS scheduling and display the results.

End of main:

Returns 0, indicating that the program has executed successfully.

Shortest Job First (Preemptive)

Header Files:

- `#include <stdio.h>`: Includes standard input/output library for input and output operations.
- `#include <limits.h>`: Includes header file for constants like `INT_MAX` used for finding minimum value.

Structure Definition:

- `struct Process`: Defines a structure to represent a process with its attributes:
- `pid`: Process ID.
- `arrival`: Arrival time of the process.
- `burst`: Burst time of the process.
- `remaining`: Remaining time of the process.
- `completion`: Completion time of the process.
- `turnaround`: Turnaround time of the process.
- `waiting`: Waiting time of the process.

Function `findShortestJob`:

- Finds the process with the shortest remaining time at a given time.
- Takes an array of processes, number of processes, and current time as input.
- Initializes `minTime` to `INT_MAX` and `shortest` to -1.
- Iterates through all processes:
- If the process has arrived (`arrival time ≤ current time`), has remaining time, and has a shorter remaining time than the current minimum, update `minTime` and `shortest`.
- Returns the index of the process with the shortest remaining time.

Function `preemptiveSJF`:

- Implements the preemptive Shortest Job First (SJF) scheduling algorithm.
- Takes an array of processes and the number of processes as input.
- Initializes `currentTime`, `completed`, `totalTurnaround`, `totalWaiting`, and `sequence` array.
- Iterates until all processes are completed:
- Finds the process with the shortest remaining time using `findShortestJob`.
- If no process is found, increment `currentTime`.
- Otherwise, increment `currentTime`, decrement the remaining time of the shortest process, and add the process ID to the `sequence` array.
- If the process is completed (remaining time is 0), update its completion time, turnaround time, and waiting time, and increment `completed`.
- Calculates and prints average turnaround time and average waiting time.

Main Function:

- Reads the number of processes and process details (arrival time and burst time).
- Initializes the remaining time of each process to its burst time.
- Calls the `preemptiveSJF` function to perform scheduling.
- Returns 0 to indicate successful execution

Shortest Job First (Non-Preemptive)

Header File:

- `#include <stdio.h>`: Includes standard input/output library for input and output operations.

Structure Definition:

- `struct Process`: Defines a structure to represent a process with its attributes:
- `pid`: Process ID.
- `arrival`: Arrival time of the process.
- `burst`: Burst time of the process.
- `completion`: Completion time of the process.
- `turnaround`: Turnaround time of the process.
- `waiting`: Waiting time of the process.
- `isCompleted`: Flag to indicate if the process is completed.

Function `sortProcesses`:

- Sorts processes based on arrival time and burst time.
- Takes an array of processes and the number of processes as input.
- Uses a bubble sort algorithm to sort the processes.
- If two processes have the same arrival time, they are sorted based on burst time.

Function `nonPreemptiveSJF`:

- Implements the non-preemptive Shortest Job First (SJF) scheduling algorithm.
- Takes an array of processes and the number of processes as input.
- Initializes `currentTime`, `completed`, `totalTurnaround`, `totalWaiting`, and `sequence` array.
- Iterates until all processes are completed:
- Finds the process with the shortest burst time among those that have arrived but not completed.
- If no process is found, increment `currentTime`.
- Otherwise, update the completion time, turnaround time, and waiting time of the shortest process, set its `isCompleted` flag to 1, and add it to the `sequence` array.
- Calculates and prints average turnaround time and average waiting time.

Main Function:

- Reads the number of processes and process details (arrival time and burst time).
- Initializes the `isCompleted` flag of each process to 0.
- Sorts the processes based on arrival time and burst time.
- Calls the `nonPreemptiveSJF` function to perform scheduling.
- Returns 0 to indicate successful execution.

Round Robin Scheduling

Structure Definition:

- struct Process: Defines a structure to represent a process with its attributes:
- pid: Process ID.
- arrival: Arrival time of the process.
- burst: Total burst time of the process.
- remaining: Remaining burst time of the process.
- completion: Completion time of the process.
- turnaround: Turnaround time of the process.
- waiting: Waiting time of the process.

Function swap:

- A helper function to swap two process structures. Used for sorting.

Function sortByArrival:

- Sorts the processes based on their arrival times using a bubble sort algorithm.

Function roundRobin:

- Implements the Round Robin scheduling algorithm.
- currentTime: Keeps track of the current time.
- completed: Counts the number of completed processes.
- sequence: An array to store the process execution sequence.
- seqIndex: Index for the sequence array.
- The main loop iterates until all processes are completed.
- It checks each process if it has arrived and still has remaining time.
- If a process is eligible, it's added to the sequence and its remaining time is updated based on the quantum.
- If a process completes in this cycle, its completion time is calculated.
- If no process is executed in a cycle, the current time is incremented.
- Calculates turnaround time and waiting time for each process.
- Displays the process execution sequence, process details, average turnaround time, and average waiting time.

Main Function:

- Reads the number of processes and their arrival and burst times.
- Sorts the processes by arrival time.
- Calls the roundRobin function to perform scheduling.
- Prints the results.

Priority Scheduling (Non preemptive)

Header Files

- `#include <stdio.h>`: Includes standard input/output library for input and output operations.
- `#include <limits.h>`: Includes header file for constants like `INT_MAX` used for finding minimum value.

Structure Definition

- `struct Process`: Defines a structure to represent a process with its attributes:
- `pid`: Process ID.
- `arrival`: Arrival time of the process.
- `burst`: Burst time of the process.
- `priority`: Priority value of the process (lower value means higher priority).
- `completion`: Completion time of the process.
- `turnaround`: Turnaround time of the process.
- `waiting`: Waiting time of the process.

Function `sortByArrivalAndPriority`

- Sorts processes based on arrival time and priority.
- If two processes have the same arrival time, they are sorted based on priority.
- Uses a bubble sort algorithm for sorting.

Function `priorityScheduling`

- Implements the priority scheduling algorithm.
- `currentTime`: Keeps track of the current time.
- `completed`: Counts the number of completed processes.
- `isCompleted`: An array to keep track of completed processes.
- `sequence`: An array to store the sequence of process execution.
- `seqIndex`: Index for the sequence array.
- The main loop iterates until all processes are completed.
- Finds the process with the highest priority (lowest priority value) that has arrived.
- If a process is found, update its completion time, turnaround time, and waiting time.
- Mark the process as completed and add it to the sequence.
- If no process is found, increment `currentTime`.
- Calculates and displays average turnaround time and average waiting time.

Main Function

- Reads the number of processes and their arrival, burst time, and priority.
- Sorts processes by arrival time and priority.
- Calls the `priorityScheduling` function to perform scheduling.
- Prints the results.

Priority Scheduling – Preemptive

Structure Definition:

- struct Process: Defines a structure to represent a process with its attributes:
- pid: Process ID.
- arrival: Arrival time of the process.
- burst: Total burst time of the process.
- priority: Priority value of the process (lower value means higher priority).
- remaining: Remaining burst time of the process.
- completion: Completion time of the process.
- turnaround: Turnaround time of the process.
- waiting: Waiting time of the process.

Function preemptivePriorityScheduling:

- Implements the preemptive priority scheduling algorithm.
- currentTime: Keeps track of the current time.
- completed: Counts the number of completed processes.
- isCompleted: An array to keep track of completed processes.
- sequence: An array to store the sequence of process execution.
- seqIndex: Index for the sequence array.
- The main loop iterates until all processes are completed.
- Finds the process with the highest priority (lowest priority value) that has arrived and has remaining time.
- If a process is found, it executes for one time unit, decrements its remaining time, and adds it to the sequence.
- If a process completes, its completion time, turnaround time, and waiting time are calculated.
- If no process is found, increment currentTime.
- Calculates and displays average turnaround time and average waiting time.

Main Function:

- Reads the number of processes and their arrival, burst time, and priority.
- Initializes the remaining burst time for each process.
- Calls the preemptivePriorityScheduling function to perform scheduling.
- Prints the results.